Best Fit Line

today we are going to write our own code for fitting a line and then compare it to differnt functions in python. We are going to take our math form Daniel C. Harris, Quantitative Chemical Analysis pages 66 and 67. Python can fit a line for you. But it is good to do a few of these functions by hand to see how they work.

To begin we are going to cheat and make our lives easier and use the numpy package. This package lets us do some array operations really easily and we won't have to do for loops. I was thinking of being mean and using all for loops. But lets take advantage of python. First lets import numpy and see what we can do.

In [19]:
```
%matplotlib ipympl
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.set_printoptions(legacy='1.25') #removes the funny printing
```

In [20]:
```
x=np.array([1.,3,4,6])
print (x)
type(x)
```

```
[1. 3. 4. 6.]
```

Out[20]: numpy.ndarray

So we could enter a list but instead of calling it a list we call it a numpy array. This is like a supercharged list.

Now we can make the y

In [21]:
```
y=np.array([2,3,4,5])
```

Now this is where numpy gets really cool. you can multiply and add your lists. This is different than array math if you have taken linear algebra. But we will be able to do that also.

In [22]:
```
print (x*y)
```

```
[ 2.  9. 16. 30.]
```

do you see what it just did? It multiplied elementwise!

In [23]:
```
print (x+y)
```

```
[ 3.  6.  8. 11.]
```

In [24]:
```
print (x-y)
```

```
[-1.  0.  0.  1.]
```

In [25]:
```
print (x/y)
```

```
[0.5 1.  1.  1.2]
```

In [26]:
```
print (x%y)
```

```
[1. 0. 0. 1.]
```

If you are doing linear algebra there are methods to do true matrix multiplication. For example the dot product.

In [27]:
```
print (np.dot(x,y))
```

```
57.0
```

In [28]:
```
print (np.sum(x))
```

```
14.0
```

In [29]:
```
print (len(x))
```

```
4
```

remember that tab is your friend. if you type np. and hit tab you will see a ton of functions we can call

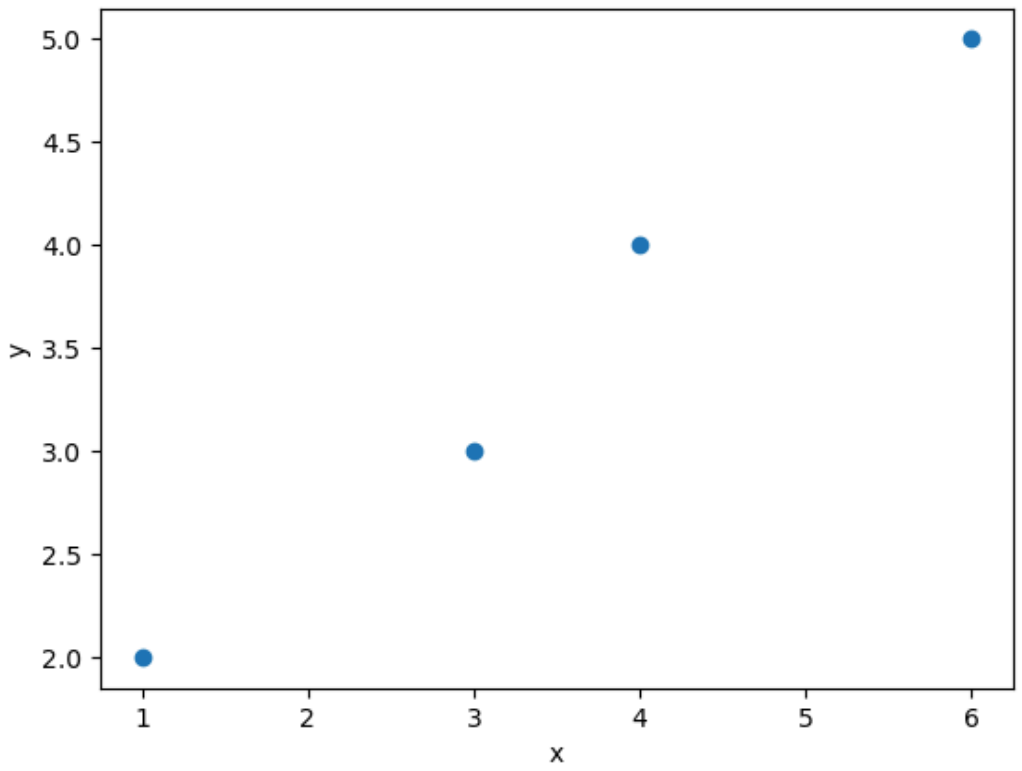
In [ ]:
```
np.
```

What does our data look like?

In [30]:
```
fig,ax=plt.subplots()
ax.scatter(x,y)
ax.set_xlabel('x')
ax.set_ylabel('y')
```

Out[30]:
```
Text(0, 0.5, 'y')
```

Figure



It is not a perfect line. So we need to fit a best fit line!

Now we can fit a line!!!!

I have now given you all the tools you need to figure out the best fit equation of a line. Remember the best fit equation of the line is y=mx+b where m is the slope and b is the intercept. We fit 2 points last time which define a line. When you have many points you have to find the best fit. We can do that! To do the fit when you have multiple points you get the equations

$$m = \frac{(n\Sigma x_i y_i - \Sigma x_i \Sigma y_i)}{(n\Sigma(x_i^2) - (\Sigma x_i)^2)}$$

$$b = \frac{\Sigma(x_i^2)\Sigma y_i - (\Sigma x_i y_i)\Sigma x_i}{n\Sigma(x_i^2) - (\Sigma x_i)^2}$$

remember x_i means for each element in the list of x.

so in our case $x_0 = 1, x_1 = 3, x_2 = 4, x_3 = 6$

if we sum up all of one list that is $\Sigma x_i = 1 + 3 + 4 + 6 = 14$

n is the length of our list

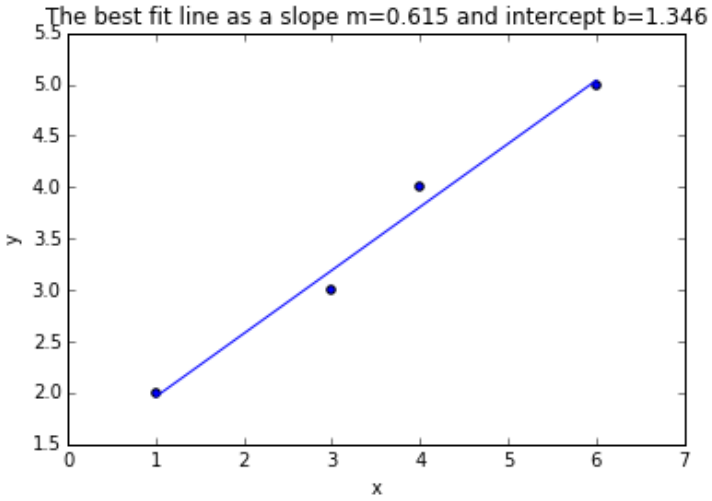
Remembr PEMDAS!

So lets plot our x and y data, look at it and then fit it.

So go ahead and figure out your m and b and then plot the line on the graph.

In [33]:

Out[33]: <matplotlib.text.Text at 0x10bff9290>



Now lets use Python to fit the line.

Two functions

1. linregress

2. Polyfit (not critical for now)

Linregress (short for linear regression)

I like linregress from scipy for my basic line fitting. the strength it has over polyfit is that it returns a p-value and an r which you can convert into r^2 along with the slope and intercept. Lets learn about it!

In [12]:
```
?stats.linregress
```

The key is we give it an x and y and then it returns:

slope : float slope of the regression line

intercept : float intercept of the regression line

r-value : float correlation coefficient

p-value : float two-sided p-value for a hypothesis test whose null hypothesis is that the slope is zero.

stderr : float Standard error of the estimate

We have talked about this some. But I want to say more explicitly here. In python when you call a function it can return many things. It doesn't have to return one number. It can return an array or multiple values. For linregress it returns 5 values. We can names these or put them in an array (I use array and list semi-interchangeably, I apologize and I will try to fix this). **HOW PYTHON CAN RETURN MANY THINGS ON THE LEFT SIDE OF AN EQUAL SIGN IS WEIRD**. Get used to it!

In [18]:
```
stats.linregress(x,y)
```

Out[18]:
```
LinregressResult(slope=np.float64(0.6153846153846154), intercept=np.float64(1.34615
38461538458), rvalue=np.float64(0.9922778767136677), pvalue=np.float64(0.0077221232
86332258), stderr=np.float64(0.05439282932204183), intercept_stderr=np.float64(0.21
414478318576893))
```

Why do I like stats.linregress? Becuase it gives us the r-value (square it and you have the r-squared) and the p-value. How do these results compare against yours that you calculated?

But if you want to use your stats results set it equal to something, it will make a list and then you can access it. Or you can set each item. so the two ways are.

First way. Set results equal to a list

In [31]:
```
stats_out=stats.linregress(x,y)
```

In [32]:
```
print(stats_out[0])
```

```
0.6153846153846154
```

In [35]:
```
stats_out
```

Out[35]:
```
LinregressResult(slope=0.6153846153846154, intercept=1.3461538461538458, rvalue=0.9
922778767136677, pvalue=0.007722123286332258, stderr=0.05439282932204183, intercept
_stderr=0.21414478318576893)
```

I just learned a cool trick that I like. You can also use dot notation with your linregress output! This is nice!

But now you can use the names to get the results.

In [36]:
```
stats_out.slope
```

Out[36]:
```
0.6153846153846154
```

In [37]:
```
print (stats_out.intercept)
```

```
1.3461538461538458
```

We can give the output meaningful names!

In [38]:
```
slope, intercept, r_value,p_value,stderr= stats.linregress(x,y)
```

In [39]:
```
slope
```

Out[39]:
```
0.6153846153846154
```

In [40]:
```
intercept
```

Out[40]:
```
1.3461538461538458
```

We can give the output nonsensical names. Remember we control the computer and we are naming them!

In [41]:
```
phineas, ferb, perry,candace,isabelle= stats.linregress(x,y)
```

In [42]:
```
print (ferb)
```

```
1.3461538461538458
```

so it is up to you on how you want to get to the data from a function like stats.linregress()

One thing I have problems with is long lines I want on multiple lines. For example sometimes I like to define a long string and then use that string as a title. To have it go over multiple lines you can use brackets. here is an example of a long string I made for a title. You can see me accessing the results both ways. Plus I added a \n to break lines and python let me break up the code on multiple lines since I was in a parantheses. Sometimes you can also add a \ to break the lines to get a line continuation. see https://stackoverflow.com/questions/4172448/is-it-possible-to-break-a-long-line-to-multiple-lines-in-python

In [43]:
```
title=('The best fit line as a slope m={:.3f} and intercept b={:.3f}'\
       .format(stats_out[0],stats_out[1])+
       '\nbest fit line linregress slope m={:.3f} and intercept b={:.3f} '\
       .format(slope,intercept))
print (title)
```

```
The best fit line as a slope m=0.615 and intercept b=1.346
best fit line linregress slope m=0.615 and intercept b=1.346
```

Now lets fix up our graph!

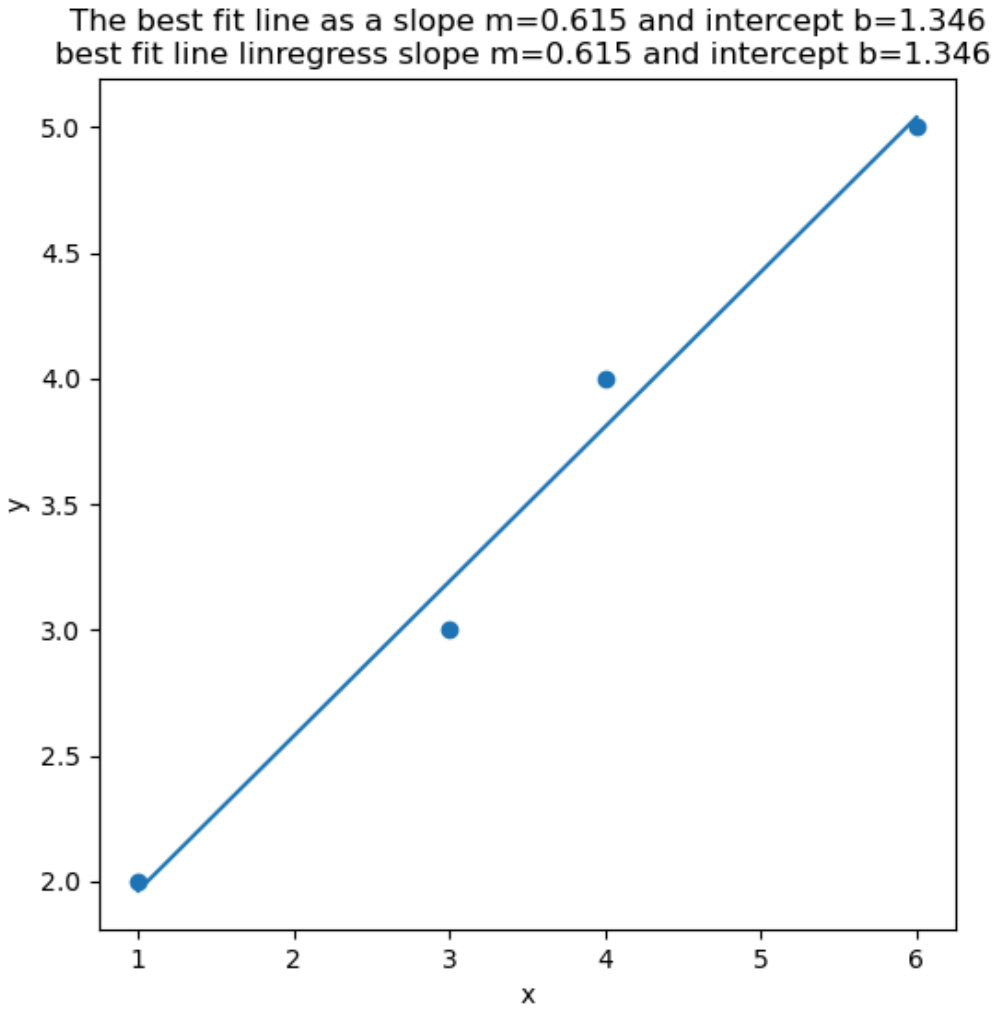
We can put the title back on.

In [44]:
```
fig,ax=plt.subplots()
fig.set_size_inches(6,6)  # I made a square graph
ax.scatter(x,y)

#find the stats

#plot the best fit line
x_fit=np.linspace(np.min(x),np.max(x))
ax.plot(x_fit,x_fit*stats_out[0]+stats_out[1])

ax.set_xlabel('x')
ax.set_ylabel('y')
title=('The best fit line as a slope m={:.3f} and intercept b={:.3f}'\
       .format(stats_out[0],stats_out[1])+
       '\nbest fit line linregress slope m={:.3f} and intercept b={:.3f} '\
```

```
        .format(slope,intercept))
ax.set_title(title)
```

Out[44]: Text(0.5, 1.0, 'The best fit line as a slope m=0.615 and intercept b=1.346\nbest fit line linregress slope m=0.615 and intercept b=1.346 ')

Figure



But I think adding a textbox to the graph makes it look more professional

Sometimes when making a graph, instead of putting in a title it looks better to put in a text box with just the details. It is a three step process to make a nice box. Scroll down this link and you can see where I got the recipe from. http://matplotlib.org/users/recipes.html It is at the bottom where it says Placing Text Boxes

1. First you define the box by making a dictionary of the box properties. We ususally call it props for the properties of the box.
2. Then you make the text string you want in the box. for a linear equation you usually want slope, interecept, r^2, and p-value
3. You then say where you want the information. This is within the ax properties since we will put it into the graph. You tell it the relative location, Then you give it the text, somemore

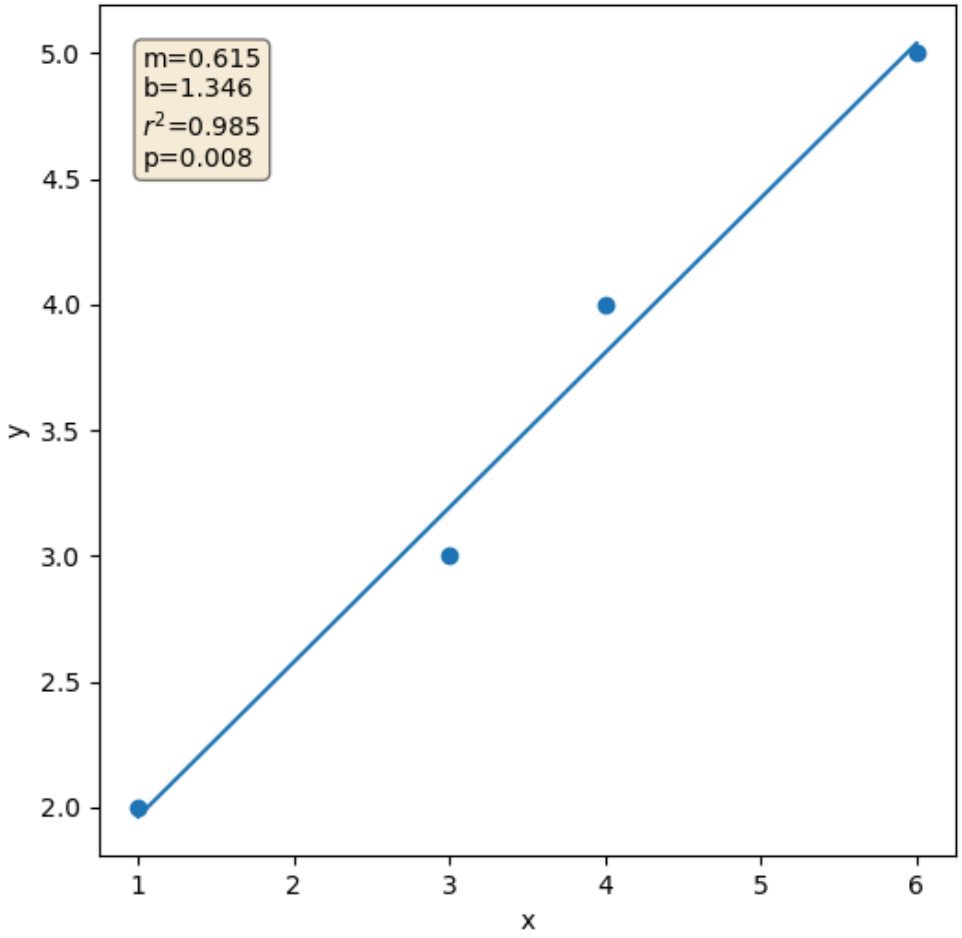
information, and then the props

4. Also add the linregress to this cell to do everything in one place to make it clean

In [45]:
```
fig,ax=plt.subplots()
fig.set_size_inches(6,6)  # I made a square graph
ax.scatter(x,y)

#Stats on the data
slope, intercept, r_value,p_value,stderr= stats.linregress(x,y)

#plot the best fit line
x_fit=np.linspace(np.min(x),np.max(x))
ax.plot(x_fit,x_fit*stats_out[0]+stats_out[1])

ax.set_xlabel('x')
ax.set_ylabel('y')

# This is the code I added to get the box below with the normal graphing
props=dict(boxstyle='round',facecolor='wheat',alpha=0.5)

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'\
        .format(slope,intercept,r_value**2, p_value)
ax.text(0.05,0.95,textstr,transform=ax.transAxes\
        ,fontsize=10,verticalalignment='top',bbox=props)
```

Out[45]:
```
Text(0.05, 0.95, 'm=0.615\nb=1.346\n$r^2$=0.985\np=0.008')
```

Figure



In [ ]:

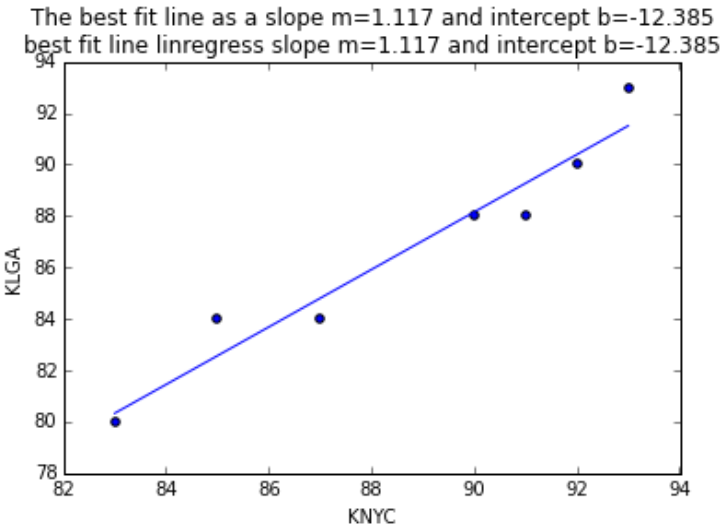
Class Assignment

Fit a line to the KLGA and Montauk data and plot it

Now can you go back and get the KNYC and KLGA weather data and see if they are correlated? I would use your program and then compare to linregress. Remember to use np.array([]) to enter the data as a numpy array. Also remember you need at least one float in your list to make it all floats. I like the second graph with the box!

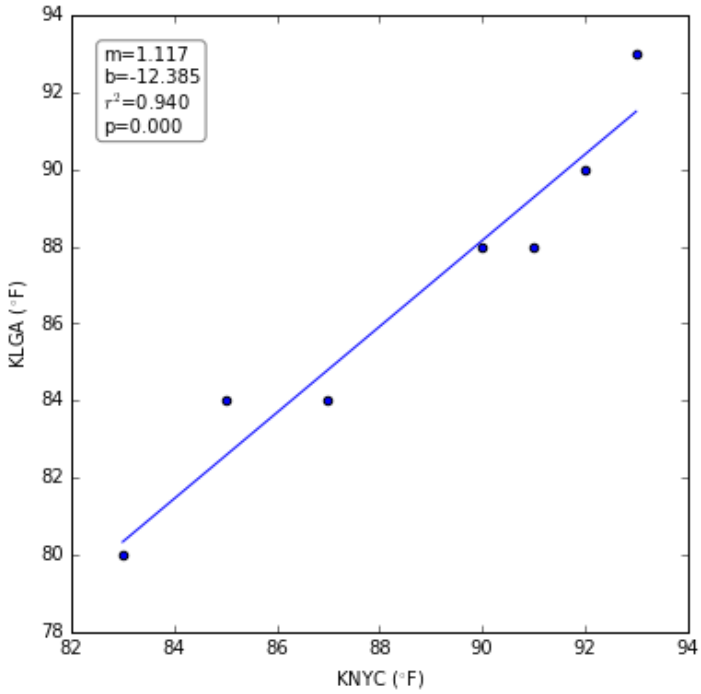
In [3]:

Out[3]: `<matplotlib.text.Text at 0x15614fd0>`



In [29]:

Out[29]: <matplotlib.text.Text at 0x9d05cf8>



This is Bonus material to help you understand p-values. If you have time left after linregress keep going with this. But not on HW.

What is a p-value?

Statisticians argue about p-values and what they exactly mean. We are going to do an exercise to help you understand.

In my simplistic world I think of a p-value as the chance of the result happening by chance.

a p-value of 0.05 means that result could happen by chance 5% of the time. It sort of but not quite means that the result is real 95% of the time.

a p-value of 0.01 means that result could happen by chance 1% of the time. It sort of but not quite means that the result is real 99% of the time.

you usually see people writing p<0.05 when they want to show the relationship is siginificant. We say it is significant because only 5% of the time it happens randomly.

This means if we randomly create data. 5% of the time we would get a p-value<0.05. 95% of the time our results would look like junk. so lets do it!

Use the nump function random to get random numbers from 0 to 1. it is np.random.random(size) and you give how many. lets do 50.

In [46]:
```
np.random.random(50)
```

Out[46]:
```
array([0.43494655, 0.18212523, 0.64058875, 0.1344949 , 0.01361897,
       0.71982023, 0.29131241, 0.70704831, 0.49865996, 0.44239256,
       0.28538041, 0.40703155, 0.80541376, 0.39869119, 0.51059245,
       0.54187569, 0.6867039 , 0.21420721, 0.87998121, 0.86427186,
       0.84706756, 0.59461833, 0.18973756, 0.44106263, 0.04924974,
       0.63475713, 0.59643627, 0.21474016, 0.8161743 , 0.7502081 ,
       0.44468346, 0.06310597, 0.32852646, 0.39601066, 0.22105364,
       0.56186312, 0.15504018, 0.61923587, 0.00742019, 0.64527563,
       0.5493542 , 0.22118838, 0.27823596, 0.55668109, 0.79909457,
       0.64948811, 0.8439744 , 0.84705429, 0.28300244, 0.51041085])
```

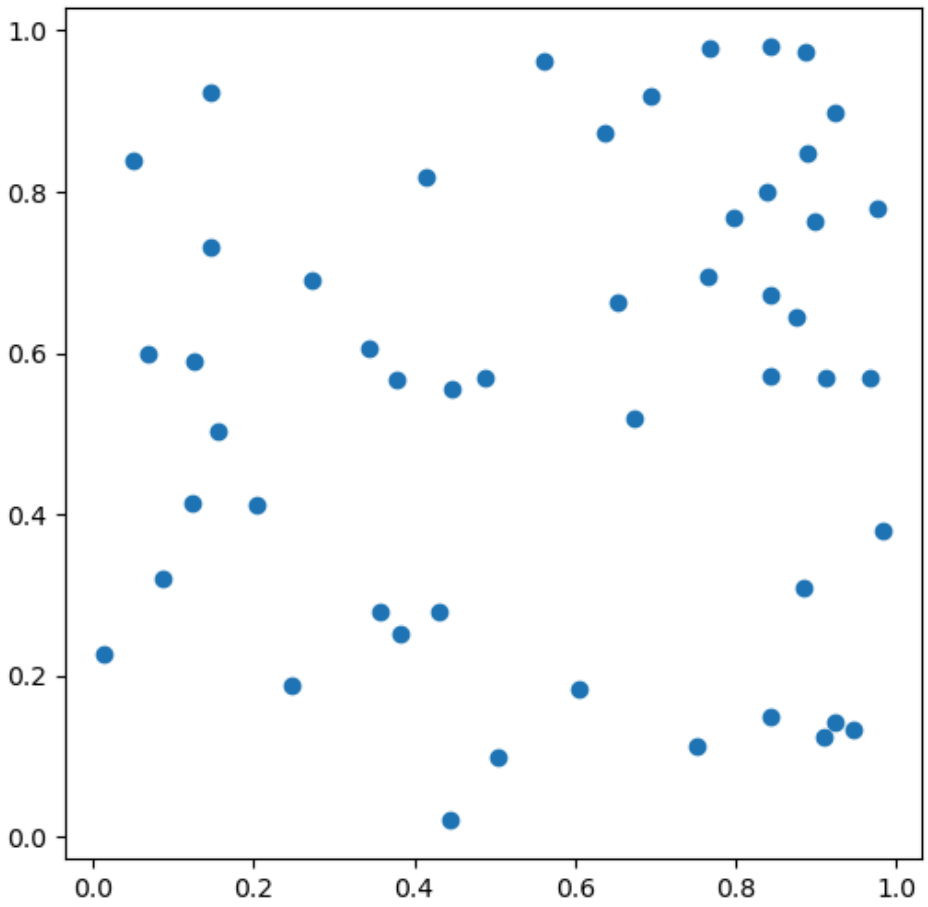
Now make your x and y data each with 50 numbers

In [47]:
```
x=np.random.random(50)
y=np.random.random(50)
```

Now plot the data. Every time you run it your data will change. Run it a few times and see if the points move around!

In [48]:
```
x=np.random.random(50)
y=np.random.random(50)

fig,ax=plt.subplots()
fig.set_size_inches(6,6)  # I made a square graph
ax.scatter(x,y)

slope, intercept, r_value,p_value,stderr= stats.linregress(x,y)
```

Figure



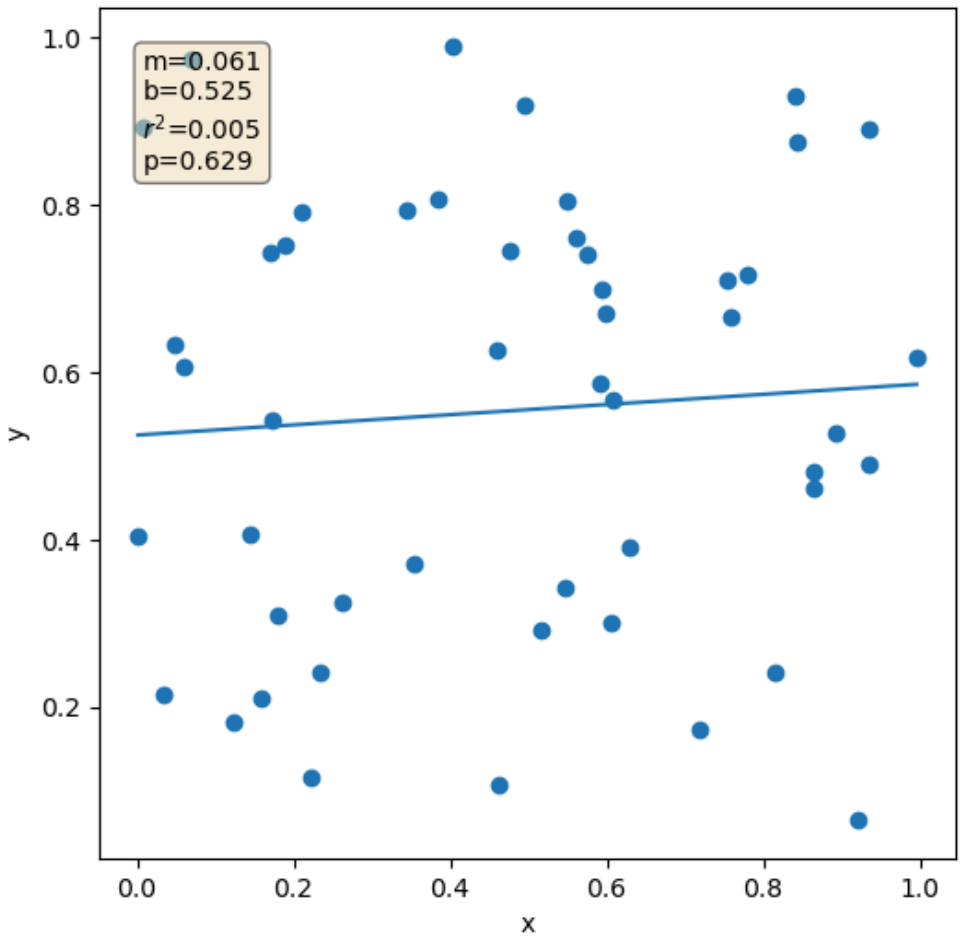
Now add the best fit line

In [49]:
```
x=np.random.random(50)
y=np.random.random(50)

fig,ax=plt.subplots()
fig.set_size_inches(6,6)  # I made a square graph
ax.scatter(x,y)

#calculate the best fit line
slope, intercept, r_value,p_value,stderr= stats.linregress(x,y)

#plot the best fit line
x_fit=np.linspace(np.min(x),np.max(x))
ax.plot(x_fit,x_fit*slope+intercept)

ax.set_xlabel('x')
ax.set_ylabel('y')

# This is the code I added to get the box below with the normal graphing
props=dict(boxstyle='round',facecolor='wheat',alpha=0.5)
textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'\
```

```
        .format(slope,intercept,r_value**2,p_value)
ax.text(0.05,0.95,textstr,transform=ax.transAxes\
        ,fontsize=10,verticalalignment='top',bbox=props)
```

Out[49]: Text(0.05, 0.95, 'm=0.061\nb=0.525\n$r^2$=0.005\np=0.629')

Figure



Now rerun the cell and count how many times it takes you to get a p-value less than 0.05. Then share with your breakout rooms It took me 27.

Now lets make the computer work for us. Think about this. We can ask the computer to make the graph above 1000 times. Then we can ask how many times we got a p-value less than 0.05. If it is random our answer should come about near but probably not exactly 50.

Now lets get rid of the graph and run the regression 1000 times and count how many times we get a significant result

In [50]:
```
num_sig=0

for i in np.arange(1000):
    x=np.random.random(50)
    y=np.random.random(50)
```

```
    #calculate the best fit line
    slope, intercept, r_value,p_value,stderr= stats.linregress(x,y)
    if p_value<0.05:
        num_sig+=1
        print('Loop Number {} with a p_value of {}'.format(i,p_value))

print('I ran the for loop 1000 times and the \
p_value was less than 0.05 {} times'.format(num_sig))
```

```
Loop Number 6 with a p_value of 0.008442175409678055
Loop Number 11 with a p_value of 0.04784186717347255
Loop Number 19 with a p_value of 0.0320088268033977
Loop Number 34 with a p_value of 0.021623520193700427
Loop Number 44 with a p_value of 0.049368414739354056
Loop Number 52 with a p_value of 0.03533960161602836
Loop Number 67 with a p_value of 0.02538334382075663
Loop Number 104 with a p_value of 0.020635171088704423
Loop Number 105 with a p_value of 0.004760665566906638
Loop Number 158 with a p_value of 0.010572687756671122
Loop Number 170 with a p_value of 0.037523858672008414
Loop Number 173 with a p_value of 0.018708861241278078
Loop Number 250 with a p_value of 0.036539864309729546
Loop Number 262 with a p_value of 0.043499622310238586
Loop Number 264 with a p_value of 0.032607588846972124
Loop Number 265 with a p_value of 0.03980420764417059
Loop Number 270 with a p_value of 0.014205752818928982
Loop Number 277 with a p_value of 0.020650841087244284
Loop Number 303 with a p_value of 0.012741296014405386
Loop Number 376 with a p_value of 0.006066711515401564
Loop Number 398 with a p_value of 0.004422544588462739
Loop Number 405 with a p_value of 0.041256530902564044
Loop Number 423 with a p_value of 0.012138876212278533
Loop Number 506 with a p_value of 0.017611551578724603
Loop Number 526 with a p_value of 0.03200855267805735
Loop Number 538 with a p_value of 0.0448041502134282
Loop Number 557 with a p_value of 0.015397633932051602
Loop Number 564 with a p_value of 0.04985816329455143
Loop Number 566 with a p_value of 0.02421649225814506
Loop Number 581 with a p_value of 0.023442184266389013
Loop Number 582 with a p_value of 0.016871090052037515
Loop Number 621 with a p_value of 0.002061962854773326
Loop Number 641 with a p_value of 0.017298053807999744
Loop Number 682 with a p_value of 0.008884868435103542
Loop Number 695 with a p_value of 0.0009011986171753818
Loop Number 696 with a p_value of 0.03046673264704424
Loop Number 697 with a p_value of 0.029565158780512645
Loop Number 720 with a p_value of 0.03035552244243557
Loop Number 747 with a p_value of 0.041870321344792065
Loop Number 753 with a p_value of 0.03911396616392059
Loop Number 763 with a p_value of 0.03942331424952338
Loop Number 767 with a p_value of 0.03461778955303397
Loop Number 776 with a p_value of 0.031116484516054085
Loop Number 795 with a p_value of 0.03181327311295768
Loop Number 800 with a p_value of 0.014783632016484622
Loop Number 822 with a p_value of 0.015108746599007709
Loop Number 842 with a p_value of 0.026015573444018242
Loop Number 891 with a p_value of 0.042437483123776897
Loop Number 909 with a p_value of 0.008179978612294605
Loop Number 943 with a p_value of 0.03557659702558512
Loop Number 949 with a p_value of 0.013052073080615375
Loop Number 954 with a p_value of 0.0017119605620574184
I ran the for loop 1000 times and the p_value was less than 0.05 52 times
```

So hopefully this helps you with a p-values. the p-value tells you how often the result may happen randomly. So the lower the p-value the lower the probability of the result happening randomly. Therefore you can "trust" the result more. But really you report the p-value so people know how you are making your choice on the significance of the results. A lot of methods report a p-value so you will be seeing this!

In [ ]: